SLIIT – Faculty of Computing

SE2030 SOFTWARE ENGINEERING

DESIGN PATTERNS – DESIGN DOCUMENT

Web-based E-Shopping Store

Group ID	WEB3G2-32
Student	GAMAACHCHIGE C D
Student ID	IT 22 6215 42
Group arrangement	Pro-rata – individual
Module	SE2030 – Software Engineering
Reference lecture	Lecture 8: Design Patterns – Part I



1. Introduction

The Web-based E-Shopping Store supports customers browsing products, placing orders and receiving order updates. This document proposes two reusable object-oriented designs based on the SE2030 lecture: Singleton (creational) and Observer (behavioral). The examples are deliberately limited to realistic responsibilities within the store.

2. Design Pattern Selection

Singleton	Use one application-wide configuration object to expose common store settings without creating multiple configuration instances.
Observer	Notify multiple interested components when an order status changes without hard-coding notification logic into order processing.

3. Singleton Pattern – StoreConfiguration

Problem: Checkout-related services may need the same tax-rate setting. Constructing unrelated configuration objects can lead to inconsistent settings within one application process.

Solution: StoreConfiguration has a private constructor, a private static instance, and a public static getInstance() method. The example uses eager initialization to avoid concurrent first-call races.

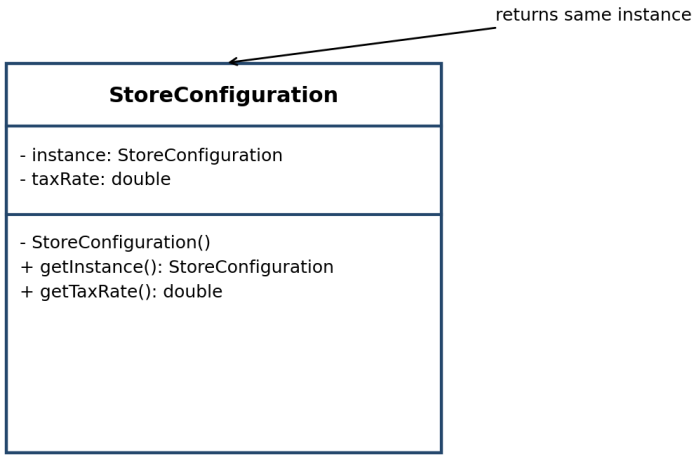


Figure 1. Proposed Singleton UML class diagram

Consequence: A single in-process configuration instance is easy to access, but global state can complicate unit testing. It does not create a single instance across multiple servers.

4. Singleton – Illustrative Java Code

```
public final class StoreConfiguration {
   private static final StoreConfiguration INSTANCE = new StoreConfiguration();
   private final double taxRate = 0.10; // illustrative only

   private StoreConfiguration() { }
```

```
    public static StoreConfiguration getInstance() {
        return INSTANCE;
    }

    public double getTaxRate() { return taxRate; }
}
// Usage: StoreConfiguration.getInstance().getTaxRate();
```

5. Observer Pattern – Order Status Notifications

Problem: After an order changes from Placed to Processing or Dispatched, more than one component may need the update. Calling every notification component directly from order logic would increase coupling.

Solution: OrderStatusManager acts as the concrete subject and maintains a collection of OrderObserver subscribers. CustomerNotifier and StaffDashboardNotifier are concrete observers that receive status updates.

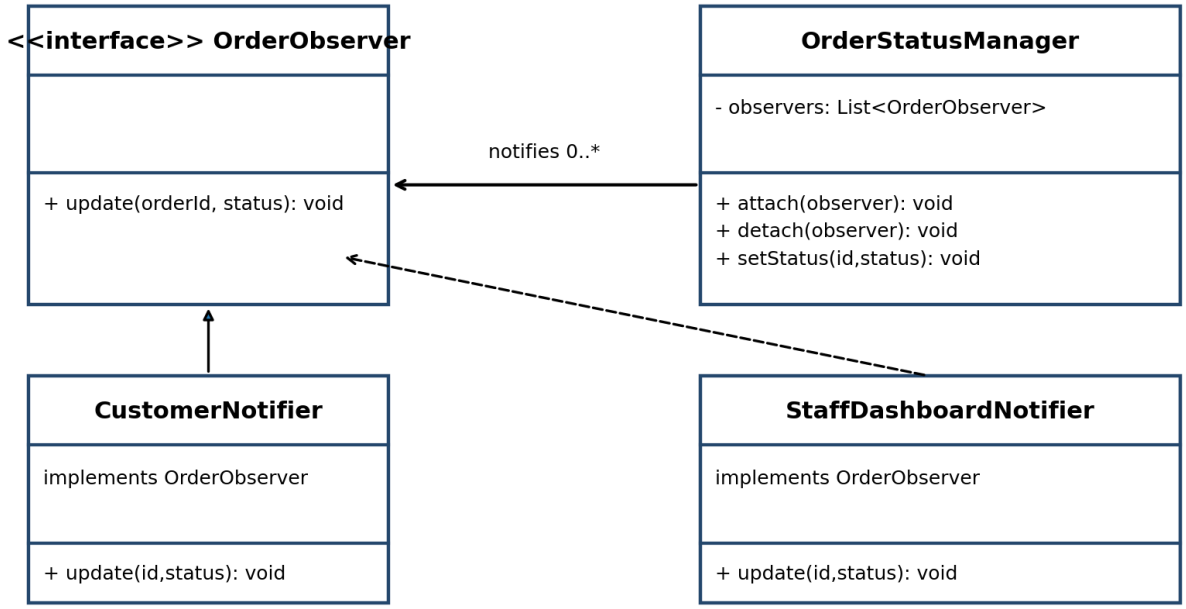


Figure 2. Proposed Observer UML class diagram

6. Observer – Interaction Scenario

1. A customer places an order and the system records its initial status.
2. The customer notification and staff dashboard components subscribe to order status updates.
3. A staff action changes the order status to Dispatched.
4. OrderStatusManager updates its state and invokes update() on each registered observer.
5. The customer notification and staff dashboard react independently.

7. Observer – Illustrative Java Code

```
import java.util.ArrayList;
import java.util.List;

interface OrderObserver {
    void update(String orderId, String status);
}
class OrderStatusManager {
```

```
    private final List<OrderObserver> observers = new ArrayList<>();
    void attach(OrderObserver observer) { observers.add(observer); }
    void detach(OrderObserver observer) { observers.remove(observer); }
    void setStatus(String orderId, String status) {
        for (OrderObserver observer : observers) {
            observer.update(orderId, status);
        }
    }
}
class CustomerNotifier implements OrderObserver {
    public void update(String id, String status) {
        System.out.println("Customer order " + id + ": " + status);
    }
}
class StaffDashboardNotifier implements OrderObserver {
    public void update(String id, String status) {
        System.out.println("Staff view: " + id + " -> " + status);
    }
}
```

8. Comparison and Design Considerations

Singleton	Creational pattern; controls instance creation. Suitable for shared immutable configuration within a process. Avoid using it as a general storage location for changing order data.
Observer	Behavioral pattern; handles one-to-many event notification. Suitable for order-status changes. Observer errors, duplicate subscriptions and delivery reliability need consideration in production.

9. Conclusion

Singleton and Observer solve different design problems in the proposed e-shopping system. StoreConfiguration centralizes shared settings, while OrderStatusManager separates status changes from the components that respond to them. These are design proposals supported by illustrative code; implementation and testing are future tasks.

10. Reference

SLIIT Faculty of Computing. SE2030 Software Engineering, Lecture 8: Design Patterns – Part I, Year 02 Semester 01. Singleton pattern: lecture pages 14–27; Observer pattern: lecture pages 28–42.